

Tipos de paginación

Guía completa con ejemplos de código y casos de uso reales

1. Paginación numérica (Offset)

■ Usado en: Google, tiendas online, resultados de búsqueda, tablas administrativas

La clásica con números de página. El usuario puede saltar directamente a cualquier página. Usa LIMIT y OFFSET en la base de datos.

Backend — Prisma

```
const productos = await prisma.producto.findMany({
  skip: (page - 1) * 10, // OFFSET
  take: 10,             // LIMIT
  orderBy: { nombre: 'asc' }
})

const total = await prisma.producto.count()
const totalPages = Math.ceil(total / 10)
```

Frontend — React

```
setPage(p => p - 1)}><←
{[1,2,3,4,5].map(n => (
  setPage(n))
  style={{ fontWeight: page === n ? 'bold' : 'normal' }}>
  {n}
)}}
setPage(p => p + 1)}>→
```

Ventajas y desventajas

- **Ventajas:** Simple, el usuario sabe dónde está, puede compartir URL de página concreta
 - **Desventajas:** Con datos que cambian frecuentemente aparecen duplicados o saltos
-

2. Scroll infinito (Cursor)

■ Usado en: Instagram, TikTok, Twitter/X, Facebook, LinkedIn feed

Los datos se cargan automáticamente al llegar al final de la página. El usuario nunca ve un botón — simplemente sigue scrolleando.

Backend — Prisma con cursor

```
const posts = await prisma.post.findMany({
  take: 10 + 1, // pedimos uno extra para saber si hay más
  cursor: cursor ? { id: cursor } : undefined,
  skip: cursor ? 1 : 0,
  orderBy: { createdAt: 'desc' }
})

const hasMore = posts.length > 10
```

```
const nextCursor = hasMore ? posts[9].id : null
```

Frontend — IntersectionObserver

```
const observerRef = useRef(null)

useEffect(() => {
  const observer = new IntersectionObserver(entries => {
    if (entries[0].isIntersecting && hasMore) cargarMas()
  })
  if (observerRef.current) observer.observe(observerRef.current)
  return () => observer.disconnect()
}, [hasMore])

return (
  <>
    {posts.map(post => <PostCard key={post.id} />)}
    <div ref={observerRef} /> { /* trigger invisible */ }
  </>
)
```

Ventajas y desventajas

- **Ventajas:** Experiencia fluida, ideal para consumo pasivo de contenido
- **Desventajas:** El usuario no puede volver a una posición concreta, difícil acceder al footer

3. Botón "Cargar más" (Cursor)

■ Usado en: YouTube (comentarios), Reddit, GitHub issues, dev.to

Similar al scroll infinito pero el usuario controla cuándo cargar. Más respetuoso con la atención del usuario.

Frontend — React

```
const [posts, setPosts] = useState([])
const [cursor, setCursor] = useState(null)
const [hasMore, setHasMore] = useState(true)
const [loading, setLoading] = useState(false)

const cargarMas = async () => {
  setLoading(true)
  const res = await fetch(`/api/posts?cursor=${cursor}&limit=10`)
  const { data, nextCursor, hasMore } = await res.json()
  setPosts(prev => [...prev, ...data])
  setCursor(nextCursor)
  setHasMore(hasMore)
  setLoading(false)
}

return (
  <>
    {posts.map(p => <PostCard key={p.id} />)}
    {hasMore && (
      <button onClick={cargarMas} disabled={loading}>
        {loading ? 'Cargando...' : 'Cargar más'}
      </button>
    )}
  </>
)
```

Ventajas y desventajas

- **Ventajas:** El usuario controla el ritmo, el footer es accesible, menos agresivo
- **Desventajas:** Requiere un clic extra cada vez

4. Paginación con URL params

■ Usado en: Blogs, e-commerce con SEO, resultados de búsqueda

Las páginas están en la URL, lo que permite compartir enlaces, hacer SEO y navegar con el botón atrás.

Next.js App Router

```
// /productos?page=3&categoria=ceramica

export default async function ProductosPage({
  searchParams
}): {
  searchParams: { page?: string; categoria?: string }
} {
  const page = Number(searchParams.page) || 1
  const limit = 12

  const productos = await prisma.producto.findMany({
    skip: (page - 1) * limit,
    take: limit,
    where: { categoria: searchParams.categoria }
  })

  return (
    <>
      <ProductosGrid productos={productos} />
      <Paginacion paginaActual={page} />
    </>
  )
}
```

Ventajas y desventajas

- **Ventajas:** URLs compartibles, SEO, funciona con el botón atrás, accesible
- **Desventajas:** Recarga en cada cambio si no usas transiciones

5. Paginación con React Query

■ Usado en: Dashboards, herramientas SaaS, apps con mucha interacción con datos

La forma moderna de manejar paginación en React — gestiona caché, estados de carga y prefetch automático de la siguiente página.

Frontend — @tanstack/react-query

```
import { useQuery, keepPreviousData } from '@tanstack/react-query'

const { data, isLoading, isPlaceholderData } = useQuery({
  queryKey: ['productos', page],
  queryFn: () => fetchProductos(page),
  placeholderData: keepPreviousData, // no parpadea al cambiar
})

// Prefetch de la siguiente página
useEffect(() => {
  if (!isPlaceholderData && data?.hasMore) {
    queryClient.prefetchQuery({
      queryKey: ['productos', page + 1],
      queryFn: () => fetchProductos(page + 1),
    })
  }
}, [data, page])
```

Ventajas y desventajas

- **Ventajas:** Caché automática, sin parpadeos, prefetch, gestión de errores integrada
- **Desventajas:** Añade dependencia externa (@tanstack/react-query)

Tabla comparativa

Tipo	Mejor para	Evitar en	Control usuario
Numérica	Catálogos, búsquedas	Feeds en tiempo real	Alto
Scroll infinito	Redes sociales, feeds	Webs con footer	Ninguno
Cargar más	Comentarios, listas	Catálogos grandes	Medio
URL params	E-commerce, SEO	Apps de una página	Alto
React Query	Dashboards, SaaS	Proyectos simples	Alto